GREATER NOIDA INSTITUTE OF TECHNOLOGY

(ENGINEERING INSTITUTE)

Plot No-7, Knowledge Park-II, Greater Noida

PROJECT SYNOPSIS

OF

CONSTA: COLLABORATIVE STUDY & PERSONAL GROWTH PLATFORM WITH VERIFIABLE OUTCOMES

Submitted in partial fulfilment for the award of the degree

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering



Submitted by:

Mohit Kumar
Roll No: 2201320100085
B.Tech CSE (Final Year)

Under the Supervision of:

Project Guide / Supervisor
Assistant Professor
Dept. of Computer Science & Engg.

Greater Noida Institute of Technology, Greater Noida
Dr. A.P.J. Abdul Kalam Technical University, Lucknow
Academic Session: 2025 – 2026

Table of Contents

Description	Page No.
Abstract	1
1. Introduction	2-4
2. Literature Review	5-6
3. Objectives	7
4. Methodology / Planning of Work	8
References	9

Abstract

Small and medium self-directed engineering student groups usually study by unstructured routines and unmonitored timers. This causes distraction and procrastination on one side, and superficial "fake productivity"—where a timer runs without genuine comprehension—on the other. This project proposes CONSTA, an integrated collaborative study and personal growth platform that converts passive study sessions into verifiable academic outcomes. In the solo study module, focus timers are decoupled from rewards; verified study experience points (XP) are awarded only upon the confirmed completion of an interactive learning objective checklist, supplemented by an integrated roadblock logger.

For group learning, the system provides structured cohorts governed by administrative roles, unique name enforcement, sub-topic checklists, and deadline timers. Coursework is managed via an eight-stage task lifecycle supporting multi-version submissions (v1 → v2 → v3) with changelog notes, multi-format attachments (handwritten photos, documents, code links), and criterion-based rubric scorecards alongside assistive AI feedback. Real-time doubt discussions are powered by WebSockets for zero-refresh message and diagram delivery, incorporating an "Accepted Solution" peer-reward mechanism (+15 XP). An auditable, append-only database transaction ledger records every XP transition to prevent point-farming exploits, complemented by a daily streak consistency heatmap. The prototype is built in Python and Node.js using React 19, Tailwind CSS v4, Express, WebSockets (ws), and native SQLite in Write-Ahead Logging (WAL) mode, validated with a 33-suite automated regression test battery.

Keywords: collaborative study, verifiable learning outcomes, rubric evaluation, multi-version submissions, real-time doubt clearing, anti-exploit gamification, WebSockets, SQLite WAL.

Chapter 1
Introduction

1.1 Background and Problem Statement

Academic productivity determines how effectively students acquire, retain, and apply complex technical concepts over time. Most self-directed learners rely on informal tools such as standalone timer applications, basic to-do checklists, or social messaging channels. Such unstructured workflows lead to two costly outcomes: passive "fake productivity," where a timer runs without genuine comprehension or problem solving, and communication fragmentation, where study tasks, reference files, and academic questions are overwhelmed by casual chat. The most common remedy in existing software is a simple Pomodoro timer or an unverified streak counter. A basic clock, however, cannot verify whether a student achieved their stated learning goals or remained idle.

Large online educational platforms emphasize structured peer collaboration, active recall, and formative rubric feedback. Platforms such as Canvas, Moodle, and Khan Academy offer institutional assignment management, while developer platforms like GitHub demonstrate the educational value of multi-version revision histories and peer review. These systems, however, are either closed within enterprise/university deployments, lack real-time peer doubt-resolution mechanics, or fail to provide student-centric gamification safeguards that reward verified effort rather than superficial clicks. No lightweight, unified platform combines outcome-verified solo study, governed study cohorts, iterative multi-version coursework submissions, and real-time doubt clearing with auditable consistency metrics.

1.2 Proposed System

This project proposes CONSTA, an integrated collaborative study and personal growth ecosystem with verifiable accountability. Here, the word "system" denotes an interconnected full-stack pipeline that captures verified study sessions, coordinates peer cohorts, manages coursework revision iterations, and streams real-time academic solutions. The platform requires students to pair focus timers with a tangible learning objective checklist; running a clock alone awards no study credentials.

In collaborative cohorts, room creators act as administrators who publish assignments with sub-topic checklists and countdown timers. When submitting coursework, students can upload multiple handwritten diagrams, documents, or repository links across successive iterations (v1, v2, v3), preserving revision notes. Submissions are evaluated against explicit rubric criteria (Correctness, Completeness, Clarity, Methodology), augmented by assistive AI evaluations providing targeted suggestions. Academic doubts are cleared via bidirectional WebSockets with zero page reloads, where peers can mark verified answers as "Accepted Solutions." All user points are strictly logged in an append-only ledger (xp_transactions) to eliminate gaming of streaks. The overall flow is shown in Fig. 1.

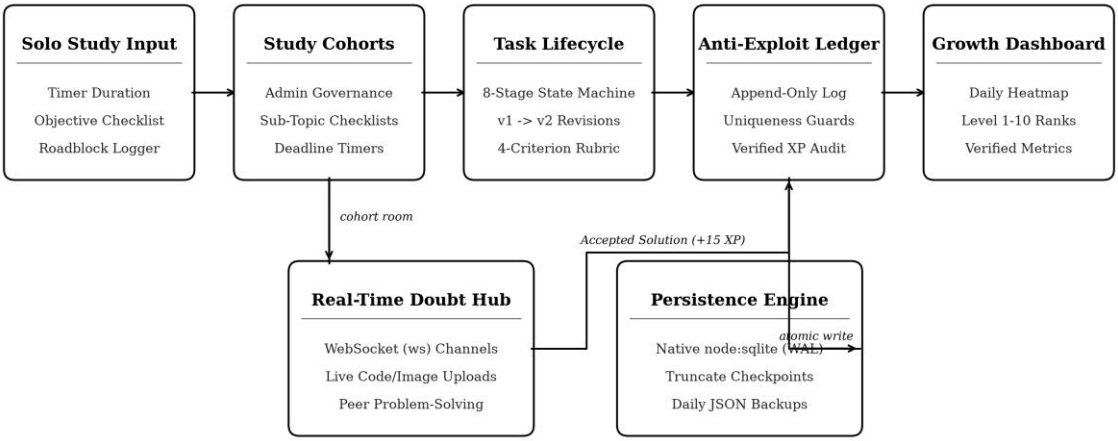


Fig. 1. Overall workflow of the proposed CONSTA Collaborative Study Platform.

Fig. 1. Overall workflow of the proposed CONSTA Collaborative Study Platform.

1.3 Field and Technology Used

The project belongs to the domain of computer-supported collaborative learning (CSCL), educational web platforms, and real-time distributed application design. It is implemented in modern JavaScript/TypeScript and Node.js. The frontend is engineered with React 19 and styled with Tailwind CSS v4 to provide a high-contrast Cream + Burgundy aesthetic; Lucide React provides standardized vector iconography. The backend service layer is built using Node.js 22 LTS and Express.js for REST routing and token-based authentication.

Real-time doubt resolution is powered by native WebSockets (ws). Data persistence is provided by the native Node.js SQLite driver (node:sqlite) running in Write-Ahead Logging (WAL) mode, guaranteeing high read concurrency without requiring external database servers. Automated unit and regression tests are executed natively using node:test and node:assert.

1.4 Outcome-Driven Verification and Anti-Exploit Gamification

Educational gamification frequently degrades when students discover exploits—such as clicking buttons repeatedly or leaving background processes active—to accumulate badges. CONSTA resolves this via outcome-driven verification and an auditable transaction ledger. A focus timer by itself does not increment the study streak or award XP. Instead, users must explicitly verify target objectives and log roadblocks encountered. Furthermore, every XP modification is recorded as a discreet record in xp_transactions containing the user ID, delta amount, source category, and entity reference ID. Duplicate awards for the same session or task are blocked at the database constraint level. When doubts are resolved, only the question author can award the "Accepted Solution" designation (+15 XP), preventing collusion and ensuring authentic peer assistance.

1.5 Key Technical Terms

The operational terms, formulas, and parameters used throughout this synopsis are summarized in Table 1.

Table 1. Key platform terminology and operational parameters.

Term	Meaning / Operational Formula
Verifiable Outcome	A concrete learning objective confirmed upon session completion; required for XP.
Roadblock Logger	A structured data store capturing conceptual obstacles, bugs, or bottlenecks during study.
Cohort Admin	The user who creates a study group; holds exclusive rights to assign tasks and manage members.
Task Lifecycle	Eight states: not_started, in_progress, submitted, under_review, revision_requested, approved, rejected, completed.
Multi-Version Revision	Iterative submission tracking (v1 → v2 → v3) preserving prior attachments, diffs, and changelogs.
Rubric Scorecard	Criterion assessment: 0.30×Correctness + 0.30×Completeness + 0.20×Clarity + 0.20×Methodology.
Accepted Solution	Author-endorsed chat response resolving an academic doubt, transferring +15 XP to solver.
Audited XP Ledger	Append-only database table (xp_transactions) logging every reward or penalty with reference keys.
Write-Ahead Logging (WAL)	SQLite concurrency mode allowing simultaneous non-blocking reads while writing safely.

Chapter 2
Literature Survey

Collaborative learning theory. Modern educational psychology demonstrates that active peer discussion and collaborative problem-solving significantly increase retention compared to isolated study [1]. Collaborative learning environments encourage students to articulate reasoning, identify conceptual gaps, and engage in cognitive restructuring [2]. Traditional Learning Management Systems (LMS), such as Blackboard or Moodle, structure institutional materials but offer rigid, top-down workflows that fail to support spontaneous student-formed cohorts or agile study groups [3].

Focus tracking and the "passive timer" fallacy. The Pomodoro technique has been widely adopted in digital productivity applications (e.g., Forest, Pomofocus) to reduce cognitive fatigue and procrastination [4]. However, existing applications treat time duration as a proxy for learning outcome [5]. Empirical studies indicate that tracking elapsed time without measuring output fosters passive clock-watching and creates an illusion of competence [6]. CONSTA addresses this limitation by decoupling clock duration from learning credentials, requiring tangible checklist confirmation.

Formative rubrics and iterative revisions. Formative feedback that occurs during the learning process is considerably more effective than summative letter grades delivered after completion [7]. Rubric-referenced evaluations provide transparent criteria regarding what constitutes mastery, guiding peer reviewers toward constructive assessments [8]. Furthermore, allowing students to resubmit work through multi-version revisions mirrors professional software engineering practices and promotes a growth mindset [9].

Real-time academic doubt resolution. Instantaneous messaging platforms have become the de facto medium for informal student interaction [10]. However, using general chat applications for academic queries leads to conversational drift, lost homework files, and unverifiable answers [11]. Dedicated question-and-answer architectures (e.g., Stack Overflow) illustrate that reputation incentives and "accepted answer" markers reliably surface high-quality technical explanations [12]. CONSTA synthesizes real-time WebSockets with accepted-answer peer rewards directly inside cohort study channels.

Anti-exploit gamification. Gamification elements—such as points, badges, and leaderboards—stimulate short-term intrinsic and extrinsic motivation [13]. Nonetheless, poorly designed point systems are susceptible to "gaming the system," wherein users optimize for points rather than learning [14]. To maintain platform credibility, game mechanics must incorporate transparent, auditable transaction ledgers, daily consistency thresholds, and penalties for exploitative behaviors [15].

Research gap. The reviewed literature confirms that existing tools (i) track time without validating learning outcomes, (ii) manage assignments without supporting iterative multi-version revisions or rubric-driven peer feedback, (iii) conduct study chats on unmoderated third-party platforms with no solution-verification mechanisms, or (iv) employ exploitable point systems lacking auditable database trails. This project addresses all four gaps in a single, responsive prototype.

References

1. D. W. Johnson, R. T. Johnson, and K. A. Smith, "Cooperative learning: Improving university instruction by basing practice on validated theory," Journal on Excellence in College Teaching, vol. 25, no. 3&4, pp. 85–118, 2014.

2. P. Dillenbourg, "What do you mean by collaborative learning?," Collaborative-learning: Cognitive and Computational Approaches, Elsevier, pp. 1–19, 1999.

3. S. Graf and Kinshuk, "Advanced adaptability in learning management systems," International Journal of Computer and Information Engineering, vol. 1, no. 8, pp. 2530–2535, 2007.

4. F. Cirillo, The Pomodoro Technique: The Acclaimed Time-Management System That Has Transformed How We Work, New York: Crown Publishing, 2018.

5. J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, "Improving students' learning with effective learning techniques," Psychological Science in the Public Interest, vol. 14, no. 1, pp. 4–58, 2013.

6. B. Oakley, A Mind for Numbers: How to Excel at Math and Science (Even If You Flunked Algebra), New York: TarcherPerigee, 2014.

7. J. Hattie and H. Timperley, "The power of feedback," Review of Educational Research, vol. 77, no. 1, pp. 81–112, 2007.

8. H. G. Andrade, "Using rubrics to promote thinking and learning," Educational Leadership, vol. 57, no. 5, pp. 13–18, 2000.

9. C. S. Dweck, Mindset: The New Psychology of Success, New York: Random House, 2006.

10. T. Tang and M. Wang, "Students' motivation for using social media in higher education," Computers & Education, vol. 146, p. 103730, 2020.

11. R. Junco, "The relationship between frequency of social messaging and academic performance," Computers & Education, vol. 59, no. 2, pp. 505–514, 2012.

12. L. Mamykina, B. Manoim, M. Mittal, G. Hripcsak, and B. Hartmann, "Design lessons from the success of Stack Overflow," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '11), 2011, pp. 2845–2854.

13. S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining gamification," in Proc. 15th Int. Academic MindTrek Conf., 2011, pp. 9–15.

14. R. S. Baker, A. T. Corbett, K. R. Koedinger, and A. Z. Wagner, "Off-task behavior in the cognitive tutor classroom: When students 'game the system'," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '04), 2004, pp. 383–390.

15. K. Seaborn and D. I. Fels, "Gamification in theory and practice: A systematic review," International Journal of Human-Computer Studies, vol. 74, pp. 14–31, 2015.

Chapter 3
Objective

Based on the literature review, the project aims to achieve the following objectives:

1. To forecast and track study productivity through an outcome-driven solo study module where study duration is decoupled from reward incentives, requiring active completion of an objective checklist and roadblock logger.

2. To implement a collaborative cohort system with role-based governance, enabling group creators to act as Admins who publish multi-topic checklists, configure duration countdown timers in days, and control group membership.

3. To convert assignment grading into an iterative coursework submission engine supporting an 8-stage lifecycle (not_started to completed) with multi-version revisions (v1 → v2 → v3), multi-page image attachments, PDFs up to 15MB, and repository links.

4. To construct a standardized rubric-driven peer review system evaluating submissions across four distinct dimensions (Correctness, Completeness, Clarity, and Methodology), accompanied by assistive AI evaluation scoring.

5. To deploy a real-time doubt-solving communication engine utilizing native WebSockets (ws) for zero-refresh message broadcasting, image/diagram sharing, and author-validated “Accepted Solution” rewards (+15 XP).

6. To design an anti-exploit gamification and streak architecture backed by an auditable, append-only database transaction ledger (xp_transactions) that records all score transitions with entity reference IDs and visualizes consistency via an activity heatmap.

7. To guarantee high database reliability and zero data loss on local and server restarts by configuring native SQLite in Write-Ahead Logging (WAL) mode with automated truncation checkpoints.

8. To validate the complete platform through automated end-to-end testing, ensuring that all 33 test suites (covering auth, cohorts, submissions, real-time WebSockets, and database persistence) pass with 100% reliability.

Chapter 4
Methodology

Step 1: System Authentication and Security Layer. A stateless authentication pipeline is implemented using Node.js and Express. Passwords are cryptographically salted and hashed using bcryptjs (salt rounds = 10). Authentication tokens are generated using JSON Web Tokens (JWT) with bearer header authorization. User profiles are initialized with isolated records, supporting username availability validation and secure email-based reset code verification.

Step 2: Outcome-Driven Solo Study Architecture. The client interface renders a configurable study session controller (subject, topic, target duration, difficulty). Active session states are maintained in local memory and synchronized with the backend. Study duration timers operate independently of reward calculations. Upon session conclusion, users must review and mark off their concrete checklist of learning objectives and log encountered roadblocks. Verified XP is credited to the database only after objective confirmation.

Step 3: Collaborative Cohorts and Administrative Governance. A relational schema establishes study cohorts with unique, case-insensitive group names. Cohort creators are automatically assigned the admin role, granting privileges to assign group tasks, configure deadline timers in days, approve private join requests, and manage membership. Cohort discovery supports both public browsing and direct peer invitation via unique join codes.

Step 4: Multi-Version Submission and Rubric Evaluation Engine. Tasks advance through an 8-stage state machine. When completing a task, learners upload multi-page handwritten answers, code links, or PDFs via Multer middleware with size safeguards (15MB max). When revisions are requested, the engine increments the submission version (v1 → v2 → v3), preserving previous iterations and changelog notes. Reviewers grade submissions using structured rubric scorecards with criterion weights, while an assistive pedagogical AI service parses submissions to output preliminary criterion scores and missing steps.

Step 5: Real-Time WebSocket Doubt Resolution. A dedicated WebSocket server (ws) binds to the HTTP server to manage real-time communication channels. Messages and photo attachments stream across active connections with sub-100ms latency without full-page reloads. The original question author can mark any peer response as the “Accepted Solution,” which automatically triggers a +15 XP grant in the audited ledger and displays an accepted badge to all room members.

Step 6: Data Persistence, Testing, and Verification. The database is configured on native node:sqlite with PRAGMA journal_mode = WAL and PRAGMA synchronous = NORMAL. Automatic checkpoints (wal_checkpoint(TRUNCATE)) prevent uncommitted write-ahead log bloating. A test suite of 33 automated tests verifies multi-user isolation, room creation, rubric grading, real-time message broadcasting, and cold-restart persistence.

Requirements. Software: Node.js (v22.5.0+), npm v10+, React 19, Vite 8, Tailwind CSS v4, Express.js, native node:sqlite, WebSockets (ws). Hardware: Standard laptop or desktop computer, 4 GB RAM, 500 MB free storage.

References

[1] S. J. Taylor and B. Letham, “Forecasting at scale,” The American Statistician, vol. 72, no. 1, pp. 37–45, 2018.

[2] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control. San Francisco, CA, USA: Holden-Day, 1970.

[3] Kaggle, “Store Item Demand Forecasting Challenge.” [Online]. Available: https://www.kaggle.com/c/demand-forecasting-kernels-only (Accessed: Sep. 10, 2026).

[4] Amazon Web Services, “Demand forecasting for retail.” [Online]. Available: https://aws.amazon.com/retail/demand-forecasting/ (Accessed: Sep. 10, 2026).

[5] E. A. Silver, D. F. Pyke, and D. J. Thomopoulos, Inventory and Production Management in Supply Chains, 4th ed. Boca Raton, FL, USA: CRC Press, 2016.

[6] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[7] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] Streamlit, “Streamlit documentation.” [Online]. Available: https://docs.streamlit.io (Accessed: Oct. 29, 2026).

[9] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” ACM Computing Surveys, vol. 46, no. 4, Art. no. 44, 2014.

[10] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” International Journal of Forecasting, vol. 22, no. 4, pp. 679–688, 2006.

[11] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “M5 accuracy competition: Results, findings, and conclusions,” International Journal of Forecasting, vol. 38, no. 4, pp. 1346–1364, 2022.

[12] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.